HW04

ENAE 601: Astrodynamics

October 21, 2026 at 23:59:00

Dr. Healy

Vai Srivastava

Problem 1

Curtis 2.45

Relative to an earth-centered, nonrotating frame, the position and velocity vectors of a spacecraft are $\mathbf{r}_0 = 6320\hat{\mathbf{i}} + 7750\hat{\mathbf{k}}$ (km) and $\mathbf{v}_0 = 11\hat{\mathbf{j}}$ (km s^{-1}). Use the $\Delta\theta$ Lagrange coefficient method.

a) Find the position vector 10 min later.

b) Calculate the change in true anomaly over the 10 min time span.

a) With $\mu = 398\,600\,\text{km}^3\,\text{s}^{-2}$, the initial radius and radial velocity are:

$$r_0 = \sqrt{6320^2 + 7750^2}\ \text{km} = 10000.244\,997\,\text{km},$$
$$v_{r_0} = \frac{\mathbf{r}_0 \cdot \mathbf{v}_0}{r_0} = 0.$$

The angular momentum, eccentricity vector, and specific energy are:

$$\mathbf{h} = \mathbf{r}_0 \times \mathbf{v}_0 = [-85250, 0, 69520]\ (\text{km}^2\,\text{s}^{-1}), \quad h = 110\,002.694\,967\,\text{km}^2\,\text{s}^{-1},$$
$$\mathbf{e} = \frac{\mathbf{v}_0 \times \mathbf{h}}{\mu} - \frac{\mathbf{r}_0}{r_0} = [1.2865302852, 0, 1.5776281188], \quad e = 2.0356990583,$$
$$\mathcal{E} = \frac{v_0^2}{2} - \frac{\mu}{r_0} = 20.640\,977\,\text{km}^2\,\text{s}^{-2}.$$

Since $e > 1$ and $\mathcal{E} > 0$, the trajectory is a hyperbola. Furthermore, $v_{r_0} = 0$ and $\mathbf{e}$ is parallel to $\mathbf{r}_0$, so the spacecraft is at perigee at $t = 0$, meaning $\theta_0 = 0$.

The $\Delta\theta$ Lagrange coefficients require the true anomaly swept in the 10 min coast, which we obtain from Kepler's equation for the hyperbola. With $t = 600\,\text{s}$ measured from perigee, the hyperbolic mean anomaly is:

$$M_h = \frac{\mu^2}{h^3}(e^2 - 1)^{\frac{3}{2}} t = 0.3992586277.$$

Solving $e\sinh(F) - F = M_h$ for the hyperbolic eccentric anomaly F via Newton's method, starting at $F_0 = M_h$:

$$F_{k+1} = F_k - \frac{e\sinh(F_k) - F_k - M_h}{e\cosh(F_k) - 1},$$

converges to $F = 0.3689340897$. Converting to true anomaly,

$$\tan\left(\frac{\theta}{2}\right) = \sqrt{\frac{e+1}{e-1}}\tanh\left(\frac{F}{2}\right),$$
$$\theta = 34.685\,070°,$$
$$\Delta\theta = \theta - \theta_0 = 34.685\,070°.$$

The orbit equation in terms of $\Delta\theta$ then gives the radius at $t = 10\,\text{min}$:

$$r = \frac{h^2}{\mu}\frac{1}{1 + \left(\frac{h^2}{\mu r_0} - 1\right)\cos(\Delta\theta) - \frac{h v_{r_0}}{\mu}\sin(\Delta\theta)} = 11353.185\,524\,\text{km}.$$

The Lagrange coefficients are:

$$f = 1 - \frac{\mu r}{h^2}(1 - \cos(\Delta\theta)) = 0.9335408947,$$
$$g = \frac{r r_0}{h}\sin(\Delta\theta) = 587.336\,691\,\text{s},$$
$$\dot{f} = \frac{\mu}{h}\frac{1-\cos(\Delta\theta)}{\sin(\Delta\theta)}\left[\frac{\mu}{h^2}(1-\cos(\Delta\theta)) - \frac{1}{r_0} - \frac{1}{r}\right] = -0.000\,206\,198\,5\ \text{s}^{-1},$$
$$\dot{g} = 1 - \frac{\mu r_0}{h^2}(1 - \cos(\Delta\theta)) = 0.9414607174.$$

Thus, the position vector 10 min later is:

$$\begin{aligned}\mathbf{r} &= f\mathbf{r}_0 + g\mathbf{v}_0 \\ &= 5899.978454\hat{\mathbf{i}} + 6460.703599\hat{\mathbf{j}} + 7234.941934\hat{\mathbf{k}}\ \text{(km)}\end{aligned}$$ ■

As a check, $|\mathbf{r}| = 11353.185\,524$ km matches the orbit equation, $f\dot{g} - \dot{f}g = 1.000000000000$, and $\mathbf{v} = \dot{f}\mathbf{r}_0 + \dot{g}\mathbf{v}_0 = [-1.303174, 10.356068, -1.598038]$ (km s^{-1}) conserves both $\mathbf{h}$ and $\mathcal{E}$. A direct RK4 integration of $\ddot{\mathbf{r}} = -\mu\mathbf{r}/r^3$ over the same 600 s also reproduces $\mathbf{r}$ to within 1 mm.

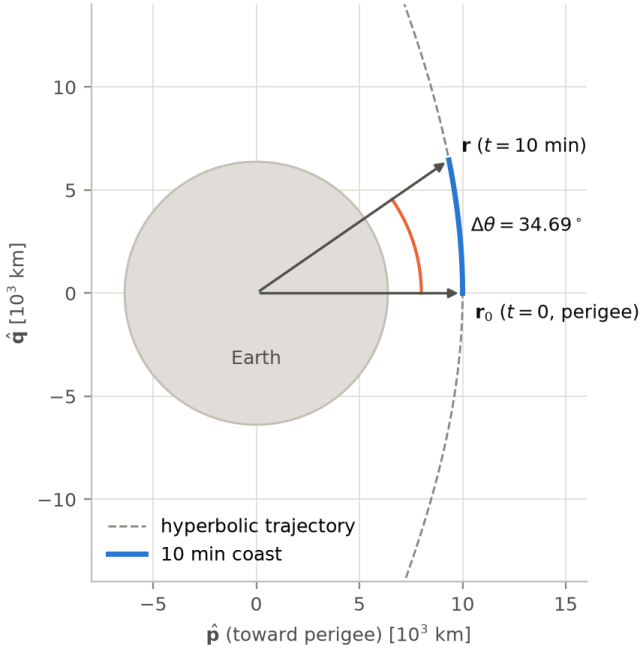


Figure 1: Hyperbolic trajectory in the perifocal frame, with the 10 min coast from perigee highlighted.

b) From part (a), the spacecraft starts at perigee ($\theta_0 = 0$) and reaches $\theta = 34.685\,070°$ after 10 min, so the change in true anomaly is:

$$\Delta\theta = \theta - \theta_0 = 34.685\,070° \quad ■$$

As an independent check that bypasses Kepler's equation and the Lagrange coefficients entirely, numerically integrating $\ddot{\mathbf{r}} = -\mu\mathbf{r}/r^3$ from $(\mathbf{r}_0, \mathbf{v}_0)$ for 600 s with RK4 gives $\mathbf{r}_{\text{RK4}} = [5899.978454, 6460.703599, 7234.941934]$ (km). The angle between the two position vectors is then:

$$\cos(\Delta\theta) = \frac{\mathbf{r}_0 \cdot \mathbf{r}_{\text{RK4}}}{r_0 |\mathbf{r}_{\text{RK4}}|} = 0.8222923550,$$

and $(\mathbf{r}_0 \times \mathbf{r}_{\text{RK4}}) \cdot \mathbf{h} > 0$, so the spacecraft swept $\Delta\theta = 34.685\,070°$ in the direction of motion, in agreement with the above.

Problem 2

Curtis 7.7

A space station is in an earth orbit with a 90 min period. At $t = 0$, a satellite has the following position and velocity components relative to a CW frame attached to the space station: $\delta\mathbf{r} = 1\hat{\mathbf{i}}$ (km) and $\delta\mathbf{v}_0 = 10\hat{\mathbf{j}}$ ($\mathrm{m\,s^{-1}}$). How far is the satellite from the space station 15 min later?

In the Clohessy–Wiltshire (CW) frame attached to the space station, $\hat{\mathbf{i}}$ is radial, $\hat{\mathbf{j}}$ is along-track, and $\hat{\mathbf{k}}$ is normal to the orbit. The frame rotates at the station's mean motion:

$$n = \frac{2\pi}{T} = \frac{2\pi}{5400\,\mathrm{s}} = 0.001\,163\,552\,8\,\mathrm{rad\,s^{-1}},$$
$$nt = \frac{2\pi}{90\,\mathrm{min}} \cdot 15\,\mathrm{min} = \frac{\pi}{3}.$$

The solution of the CW equations is:

$$\delta\mathbf{r}(t) = \Phi_{rr}(t)\delta\mathbf{r}_0 + \Phi_{rv}(t)\delta\mathbf{v}_0,$$
$$\delta\mathbf{v}(t) = \Phi_{vr}(t)\delta\mathbf{r}_0 + \Phi_{vv}(t)\delta\mathbf{v}_0,$$

with the state transition matrices:

$$\Phi_{rr} = \begin{bmatrix} 4-3\cos(nt) & 0 & 0 \\ 6(\sin(nt)-nt) & 1 & 0 \\ 0 & 0 & \cos(nt) \end{bmatrix}, \qquad \Phi_{rv} = \begin{bmatrix} \frac{\sin(nt)}{n} & \frac{2(1-\cos(nt))}{n} & 0 \\ \frac{2(\cos(nt)-1)}{n} & \frac{4\sin(nt)-3nt}{n} & 0 \\ 0 & 0 & \frac{\sin(nt)}{n} \end{bmatrix},$$

$$\Phi_{vr} = \begin{bmatrix} 3n\sin(nt) & 0 & 0 \\ 6n(\cos(nt)-1) & 0 & 0 \\ 0 & 0 & -n\sin(nt) \end{bmatrix}, \quad \Phi_{vv} = \begin{bmatrix} \cos(nt) & 2\sin(nt) & 0 \\ -2\sin(nt) & 4\cos(nt)-3 & 0 \\ 0 & 0 & \cos(nt) \end{bmatrix}.$$

Here $\delta\mathbf{r}_0 = [1, 0, 0]$ (km) and $\delta\mathbf{v}_0 = [0, 0.010, 0]$ ($\mathrm{km\,s^{-1}}$), so only δx_0 and $\delta\dot{y}_0$ are nonzero. Thus, only the first column of Φ_{rr} and the second column of Φ_{rv} contribute to the position. At $nt = \frac{\pi}{3}$:

$$\begin{aligned} \delta x &= \left(4 - 3\cos\left(\frac{\pi}{3}\right)\right)\delta x_0 + \frac{2\left(1-\cos\left(\frac{\pi}{3}\right)\right)}{n}\delta\dot{y}_0 \\ &= \frac{5}{2}(1\,\mathrm{km}) + (859.436\,693\,\mathrm{s})(0.010\,\mathrm{km\,s^{-1}}) \\ &= 11.094\,367\,\mathrm{km}, \\ \delta y &= 6\left(\sin\left(\frac{\pi}{3}\right) - \frac{\pi}{3}\right)\delta x_0 + \frac{4\sin\left(\frac{\pi}{3}\right) - \pi}{n}\delta\dot{y}_0 \\ &= (-1.087033)(1\,\mathrm{km}) + (277.176\,035\,\mathrm{s})(0.010\,\mathrm{km\,s^{-1}}) \\ &= 1.684\,727\,\mathrm{km}, \\ \delta z &= 0. \end{aligned}$$

Thus, the distance between the satellite and the space station 15 min later is:

$$d = |\delta\mathbf{r}| = \sqrt{\delta x^2 + \delta y^2 + \delta z^2} = 11.221\,554\,\mathrm{km} \quad \blacksquare$$

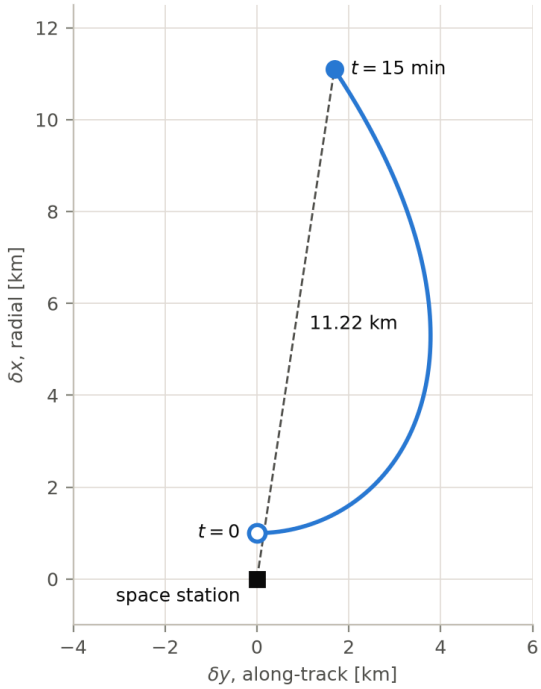


Figure 2: Satellite motion relative to the space station over the 15 min interval.

Problem 3

Curtis 7.8

Spacecraft A and B are in the same circular earth orbit with a period of 2 h. B is 6 km ahead of A. At $t = 0$, B applies an in-track Δv (retrofire) of $3\,\mathrm{m\,s^{-1}}$. Using a CW frame attached to A, determine the distance between A and B at $t = 30\,\mathrm{min}$ and the velocity of B relative to A at that instant.

In the CW frame attached to A, the mean motion and elapsed angle are:

$$n = \frac{2\pi}{T} = \frac{2\pi}{7200\,\mathrm{s}} = 0.000\,872\,664\,6\,\mathrm{rad\,s^{-1}},$$
$$nt = \frac{2\pi}{2\,\mathrm{h}} \cdot 30\,\mathrm{min} = \frac{\pi}{2}.$$

Since A and B share the same circular orbit, B is stationary in A's CW frame before the burn: $\delta\mathbf{r}_0 = 6\hat{\mathbf{j}}$ (km) and $\delta\mathbf{v}_0^- = \mathbf{0}$. Indeed, with $\delta x_0 = 0$ and $\delta\mathbf{v}_0 = \mathbf{0}$, the CW solution gives $\delta\mathbf{r}(t) = \delta\mathbf{r}_0$ for all t.

The retrofire acts opposite to the direction of motion ($-\hat{\mathbf{j}}$), so immediately after the burn:

$$\delta\mathbf{v}_0^+ = \delta\mathbf{v}_0^- + \Delta\mathbf{v} = -3\hat{\mathbf{j}}\ (\mathrm{m\,s^{-1}}) = -0.003\hat{\mathbf{j}}\ (\mathrm{km\,s^{-1}}).$$

With $\delta x_0 = 0$ and $\delta\dot{x}_0^+ = 0$, only the second columns of the state transition matrices contribute. The second column of Φ_{vr} is zero, so δy_0 does not affect the relative velocity. At $nt = \frac{\pi}{2}$, where $\sin(nt) = 1$ and $\cos(nt) = 0$:

$$\delta x = \frac{2(1-\cos(nt))}{n}\delta\dot{y}_0^+ = (2291.831\,181\,\mathrm{s})(-0.003\,\mathrm{km\,s^{-1}}) = -6.875\,494\,\mathrm{km},$$
$$\delta y = \delta y_0 + \frac{4\sin(nt) - 3nt}{n}\delta\dot{y}_0^+$$
$$= 6\,\mathrm{km} + (-816.337\,639\,\mathrm{s})(-0.003\,\mathrm{km\,s^{-1}}) = 8.449\,013\,\mathrm{km},$$
$$\delta\dot{x} = 2\sin(nt)\delta\dot{y}_0^+ = 2(-3\,\mathrm{m\,s^{-1}}) = -6\,\mathrm{m\,s^{-1}},$$
$$\delta\dot{y} = (4\cos(nt) - 3)\delta\dot{y}_0^+ = -3(-3\,\mathrm{m\,s^{-1}}) = 9\,\mathrm{m\,s^{-1}},$$

with $\delta z = \delta\dot{z} = 0$.

Thus, the distance between A and B at $t = 30\,\mathrm{min}$ is:

$$d = |\delta\mathbf{r}| = \sqrt{\delta x^2 + \delta y^2} = 10.893\,036\,\mathrm{km} \quad \blacksquare$$

The velocity of B relative to A (measured in the CW frame) at that instant is:

$$\delta\mathbf{v} = -6\hat{\mathbf{i}} + 9\hat{\mathbf{j}}\ (\mathrm{m\,s^{-1}}),$$
$$|\delta\mathbf{v}| = \sqrt{6^2 + 9^2}\ \ \mathrm{m\,s^{-1}} = 10.816\,654\,\mathrm{m\,s^{-1}} \quad \blacksquare$$

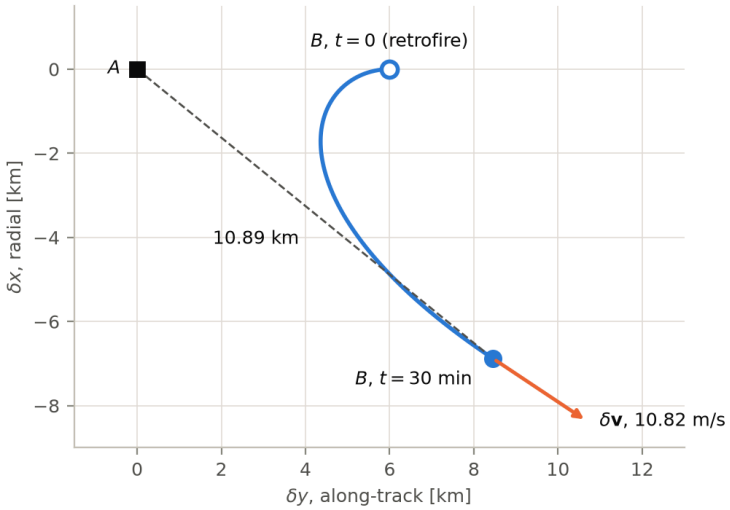


Figure 3: Motion of B relative to A after the retrofire, with the relative velocity at $t = 30\,\text{min}$.

Problem 4

Curtis 7.10

At time $t = 0$, a particle is at the origin of a CW frame with a relative velocity $\delta\mathbf{v}_0 = v\hat{\mathbf{j}}$. What will be the relative speed of the particle after a time equal to one-half the orbital period of the CW frame?

The particle starts at the origin, so $\delta\mathbf{r}_0 = \mathbf{0}$ and only the Φ_{vv} term of the CW solution survives in the relative velocity:

$$\delta\mathbf{v}(t) = \Phi_{vv}(t)\delta\mathbf{v}_0 = \begin{bmatrix} \cos(nt) & 2\sin(nt) & 0 \\ -2\sin(nt) & 4\cos(nt) - 3 & 0 \\ 0 & 0 & \cos(nt) \end{bmatrix} \begin{bmatrix} 0 \\ v \\ 0 \end{bmatrix} = v \begin{bmatrix} 2\sin(nt) \\ 4\cos(nt) - 3 \\ 0 \end{bmatrix}.$$

One-half of the orbital period is $t = \frac{T}{2} = \frac{\pi}{n}$, so $nt = \pi$, where $\sin(\pi) = 0$ and $\cos(\pi) = -1$:

$$\delta\mathbf{v}\left(\frac{T}{2}\right) = v \begin{bmatrix} 2\sin(\pi) \\ 4\cos(\pi) - 3 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ -7v \\ 0 \end{bmatrix}.$$

Thus, the relative speed after one-half of the orbital period is:

$$\left|\delta\mathbf{v}\left(\frac{T}{2}\right)\right| = 7|v| \quad \blacksquare$$

i.e., seven times the initial relative speed, now directed along $-\hat{\mathbf{j}}$.

More generally, the relative speed is:

$$|\delta\mathbf{v}| = |v|\sqrt{4\sin^2(nt) + (4\cos(nt) - 3)^2} = |v|\sqrt{13 - 24\cos(nt) + 12\cos^2(nt)},$$

whose radicand has derivative $24\sin(nt)(1 - \cos(nt)) \geq 0$ on $0 \leq nt \leq \pi$. The speed therefore grows monotonically from $|v|$ to its maximum of $7|v|$ at the half period. At that instant, $\delta\mathbf{r} = \Phi_{rv}\delta\mathbf{v}_0 = (v/n)[4, -3\pi, 0]$.

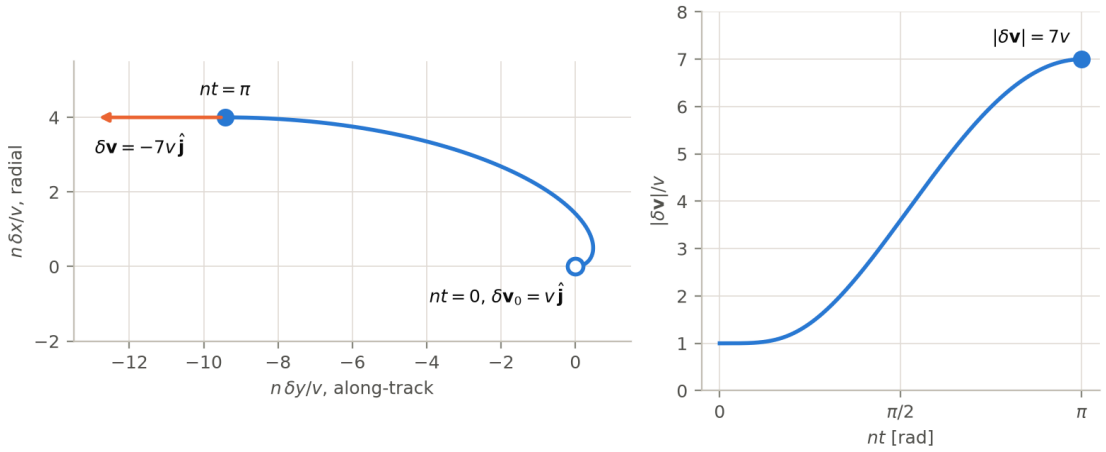


Figure 4: Nondimensional relative trajectory (left) and relative speed (right) over one-half of the orbital period.

Problem 5

Orbit Propagation with STM

a) Instructions

b) Instructions

a) Answer

b) Answer

c) Answer

./src/s05c.py Python

```python
import numpy as np
import astropy.units as u
import tellurion as tell

r1 = np.array([5740.13268349, 3314.06715, 0.])
v1 = np.array([-2.75082684, 4.76457184, 5.5016536])
```

Problem 6

Lambert Problem

Instructions

a) Instructions

b) Instructions

c) Instructions

d) Instructions

a) Answer

b) Answer

c) Answer

./src/s06c.py Python

```python
import numpy as np
import astropy.units as u
import tellurion as tell

now = tell.abstime(0.0)
r1 = tell.pvtcart(np.array([-6991, -5118, 7078]), now)
print(r1.cartesian)

r2 = tell.pvtcart(np.array([11326, -6317, -6677]), now + 5200*u.s)
print(r2.cartesian)
```

a) Answer

./src/s06d.py Python

```python
import numpy as np
import astropy.units as u
import tellurion as tell

now = tell.abstime(0.0)
r1 = tell.pvtcart(np.array([-6991, -5118, 7078]), now)
print(r1.cartesian)

r2 = tell.pvtcart(np.array([11326, -6317, -6677]), now + 5200*u.s)
print(r2.cartesian)
```

Code

./src/index.py Python

```python
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

MU_EARTH = 398600.0  # km^3/s^2
R_EARTH = 6378.0  # km
TAU = 2.0 * np.pi
ROOT = Path(__file__).resolve().parents[1]
TEXT_DIR = ROOT / "outputs" / "text"
FIGURE_DIR = ROOT / "outputs" / "figures"

# Figure ink and categorical series colors (fixed order).
INK = "#0b0b0b"
INK_SECONDARY = "#52514e"
MUTED = "#898781"
GRID = "#e1e0d9"
BASELINE = "#c3c2b7"
SERIES = ("#2a78d6", "#eb6834", "#1baf7a")

plt.rcParams.update(
    {
        "font.size": 9,
        "axes.edgecolor": BASELINE,
        "axes.labelcolor": INK_SECONDARY,
        "axes.spines.top": False,
        "axes.spines.right": False,
        "axes.grid": True,
        "axes.axisbelow": True,
        "grid.color": GRID,
        "grid.linewidth": 0.6,
        "xtick.color": MUTED,
        "ytick.color": MUTED,
        "xtick.labelcolor": INK_SECONDARY,
        "ytick.labelcolor": INK_SECONDARY,
        "legend.frameon": False,
        "savefig.dpi": 200,
    }
)


def heading(problem: str) -> None:
    """Print the heading used to separate each problem's output."""
    print(f"\n------\n {problem}\n------")
```

```
45
46
47  def clean(value: float) -> float:
48      """Zero out round-off so values like -1e-17 do not print as -0.000000."""
49      return 0.0 if abs(value) < 1e-12 else float(value)
50
51
52  def write_output(name: str, lines: list[str]) -> None:
53      """Print and save a solution's text output for inclusion in Typst."""
54      output = "\n".join(lines) + "\n"
55      print(output, end="")
56      TEXT_DIR.mkdir(parents=True, exist_ok=True)
57      (TEXT_DIR / f"s{name}.txt").write_text(output, encoding="utf-8")
58
59
60  def save_figure(fig: plt.Figure, name: str) -> None:
61      """Save a solution's figure for inclusion in Typst."""
62      FIGURE_DIR.mkdir(parents=True, exist_ok=True)
63      path = FIGURE_DIR / f"s{name}.png"
64      fig.savefig(path, bbox_inches="tight")
65      plt.close(fig)
66      print(f"saved {path.relative_to(ROOT)}")
67
68
69  def vector_lines(
70      name: str, vector: np.ndarray, unit: str = "", precision: int = 6
71  ) -> list[str]:
72      """Format each component of a vector as its own key/value line."""
73      return [
74          f"{name}_{axis} = {clean(value):.{precision}f} {unit}".rstrip()
75          for axis, value in zip("xyz", vector)
76      ]
77
78
79  def rk4(rate, state: np.ndarray, duration: float, steps: int) -> np.ndarray:
80      """Integrate state' = rate(state) with fixed-step fourth-order Runge-Kutta."""
81      dt = duration / steps
82      for _ in range(steps):
83          k1 = rate(state)
84          k2 = rate(state + dt / 2.0 * k1)
85          k3 = rate(state + dt / 2.0 * k2)
86          k4 = rate(state + dt * k3)
87          state = state + dt / 6.0 * (k1 + 2.0 * k2 + 2.0 * k3 + k4)
88      return state
89
90
```

```
91  def two_body_rate(state: np.ndarray) -> np.ndarray:
92      """Two-body equations of motion, r'' = -mu r / r^3."""
93      r, v = state[:3], state[3:]
94      return np.concatenate((v, -MU_EARTH * r / np.linalg.norm(r) ** 3))
95
96
97  def cw_rate(n: float):
98      """Linearized Clohessy-Wiltshire equations (x radial, y along-track)."""
99
100     def rate(state: np.ndarray) -> np.ndarray:
101         x, _, z, vx, vy, vz = state
102         return np.array(
103             [vx, vy, vz, 3.0 * n * n * x + 2.0 * n * vy, -2.0 * n * vx, -n * n * z]
104         )
105
106     return rate
107
108
109 def solve_hyperbolic_kepler(e: float, mean_anomaly: float) -> float:
110     """Solve e sinh(F) - F = M_h for the hyperbolic eccentric anomaly F."""
111     anomaly = mean_anomaly
112     for _ in range(50):
113         step = (e * np.sinh(anomaly) - anomaly - mean_anomaly) / (
114             e * np.cosh(anomaly) - 1.0
115         )
116         anomaly -= step
117         if abs(step) < 1e-14:
118             return anomaly
119     raise RuntimeError("Hyperbolic Kepler iteration did not converge.")
120
121
122 def cw_stm(n: float, t: float) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
123     """Clohessy-Wiltshire state transition matrices (Curtis Eq. 7.53)."""
124     s, c, nt = np.sin(n * t), np.cos(n * t), n * t
125     phi_rr = np.array(
126         [[4.0 - 3.0 * c, 0.0, 0.0], [6.0 * (s - nt), 1.0, 0.0], [0.0, 0.0, c]]
127     )
128     phi_rv = np.array(
129         [
130             [s / n, 2.0 * (1.0 - c) / n, 0.0],
131             [2.0 * (c - 1.0) / n, (4.0 * s - 3.0 * nt) / n, 0.0],
132             [0.0, 0.0, s / n],
133         ]
134     )
135     phi_vr = np.array(
136         [[3.0 * n * s, 0.0, 0.0], [6.0 * n * (c - 1.0), 0.0, 0.0], [0.0, 0.0, -n * s]]
```

```
137     )
138     phi_vv = np.array(
139         [[c, 2.0 * s, 0.0], [-2.0 * s, 4.0 * c - 3.0, 0.0], [0.0, 0.0, c]]
140     )
141     return phi_rr, phi_rv, phi_vr, phi_vv
142
143
144 def cw_propagate(
145     n: float, t: float, dr0: np.ndarray, dv0: np.ndarray
146 ) -> tuple[np.ndarray, np.ndarray]:
147     """Relative position and velocity at time t from the CW STM."""
148     phi_rr, phi_rv, phi_vr, phi_vv = cw_stm(n, t)
149     return phi_rr @ dr0 + phi_rv @ dv0, phi_vr @ dr0 + phi_vv @ dv0
150
151
152 def check_cw(n: float, t: float, dr0: np.ndarray, dv0: np.ndarray) -> float:
153     """Compare the STM solution against direct integration of the CW equations."""
154     dr, dv = cw_propagate(n, t, dr0, dv0)
155     numerical = rk4(cw_rate(n), np.concatenate((dr0, dv0)), t, 6000)
156     error = np.linalg.norm(numerical - np.concatenate((dr, dv)))
157     assert error < 1e-9, error
158     return error
159
160
161 def problem_01() -> None:
162     """Curtis 2.45: delta-theta Lagrange coefficients over a 10 min coast."""
163     heading("p01")
164     r0_vec = np.array([6320.0, 0.0, 7750.0])  # km
165     v0_vec = np.array([0.0, 11.0, 0.0])  # km/s
166     dt = 10.0 * 60.0  # s
167
168     r0 = np.linalg.norm(r0_vec)
169     v0 = np.linalg.norm(v0_vec)
170     vr0 = r0_vec @ v0_vec / r0
171     h_vec = np.cross(r0_vec, v0_vec)
172     h = np.linalg.norm(h_vec)
173     h_hat = h_vec / h
174     e_vec = np.cross(v0_vec, h_vec) / MU_EARTH - r0_vec / r0
175     e = np.linalg.norm(e_vec)
176     energy = v0**2 / 2.0 - MU_EARTH / r0
177     if e <= 1.0 or energy <= 0.0:
178         raise ValueError("Expected a hyperbolic trajectory.")
179
180     # Signed initial true anomaly; v_r0 = 0 places the spacecraft at perigee.
181     theta0 = np.arctan2(h_hat @ np.cross(e_vec, r0_vec), e_vec @ r0_vec)
182
```

```
    # Hyperbolic Kepler's equation turns the 10 min coast into delta theta.
    ratio = np.sqrt((e + 1.0) / (e - 1.0))
    anomaly0 = 2.0 * np.arctanh(np.tan(theta0 / 2.0) / ratio)
    mean_rate = MU_EARTH**2 / h**3 * (e * e - 1.0) ** 1.5  # rad/s
    mean_anomaly = e * np.sinh(anomaly0) - anomaly0 + mean_rate * dt
    anomaly = solve_hyperbolic_kepler(e, mean_anomaly)
    theta = 2.0 * np.arctan(ratio * np.tanh(anomaly / 2.0))
    dtheta = theta - theta0

    # Lagrange coefficients in terms of delta theta (Curtis Eqs. 2.152, 2.158).
    c, s = np.cos(dtheta), np.sin(dtheta)
    r = (h**2 / MU_EARTH) / (
        1.0 + (h**2 / (MU_EARTH * r0) - 1.0) * c - h * vr0 / MU_EARTH * s
    )
    f = 1.0 - MU_EARTH * r / h**2 * (1.0 - c)
    g = r * r0 / h * s
    f_dot = (
        MU_EARTH
        / h
        * (1.0 - c)
        / s
        * (MU_EARTH / h**2 * (1.0 - c) - 1.0 / r0 - 1.0 / r)
    )
    g_dot = 1.0 - MU_EARTH * r0 / h**2 * (1.0 - c)
    r_vec = f * r0_vec + g * v0_vec
    v_vec = f_dot * r0_vec + g_dot * v0_vec

    # Independent checks: invariants, the Lagrange identity, the swept angle, and
    # direct numerical integration of the two-body equations.
    np.testing.assert_allclose(np.linalg.norm(r_vec), r, rtol=1e-12)
    np.testing.assert_allclose(f * g_dot - f_dot * g, 1.0, atol=1e-12)
    np.testing.assert_allclose(np.cross(r_vec, v_vec), h_vec, rtol=0.0, atol=1e-7)
    np.testing.assert_allclose(v_vec @ v_vec / 2.0 - MU_EARTH / r, energy, rtol=1e-12)
    swept = np.arctan2(h_hat @ np.cross(r0_vec, r_vec), r0_vec @ r_vec)
    np.testing.assert_allclose(swept, dtheta, atol=1e-12)
    numerical = rk4(two_body_rate, np.concatenate((r0_vec, v0_vec)), dt, 6000)
    rk4_error = np.linalg.norm(numerical[:3] - r_vec)
    # The angle to the integrated position checks delta theta without f and g.
    r_numerical = numerical[:3]
    cos_numerical = r0_vec @ r_numerical / (r0 * np.linalg.norm(r_numerical))
    swept_numerical = np.arctan2(
        h_hat @ np.cross(r0_vec, r_numerical), r0_vec @ r_numerical
    )
    np.testing.assert_allclose(swept_numerical, dtheta, atol=1e-9)
    assert rk4_error < 1e-6, rk4_error

```

```
    write_output(
        "01a",
        [
            f"r0 = {r0:.6f} km",
            f"v0 = {v0:.6f} km/s",
            f"vr0 = {clean(vr0):.6f} km/s",
            *vector_lines("h", h_vec, "km^2/s"),
            f"h = {h:.6f} km^2/s",
            *vector_lines("e", e_vec, precision=10),
            f"e = {e:.10f}",
            f"energy = {energy:.6f} km^2/s^2",
            f"theta0 = {np.degrees(clean(theta0)):.6f} deg",
            f"Mh rate = {mean_rate:.10f} rad/s",
            f"M_h = {mean_anomaly:.10f}",
            f"F = {anomaly:.10f}",
            f"theta = {np.degrees(theta):.6f} deg",
            f"dtheta = {np.degrees(dtheta):.6f} deg",
            f"r = {r:.6f} km",
            f"f = {f:.10f}",
            f"g = {g:.6f} s",
            f"f_dot = {f_dot:.10f} 1/s",
            f"g_dot = {g_dot:.10f}",
            *vector_lines("r", r_vec, "km"),
            *vector_lines("v", v_vec, "km/s"),
            f"lagrange identity = {f * g_dot - f_dot * g:.12f}",
            f"rk4 position difference = {rk4_error:.3e} km",
        ],
    )
    write_output(
        "01b",
        [
            f"theta0 = {np.degrees(clean(theta0)):.6f} deg",
            f"theta = {np.degrees(theta):.6f} deg",
            f"dtheta = {np.degrees(dtheta):.6f} deg",
            *vector_lines("r_rk4", r_numerical, "km"),
            f"cos dtheta rk4 = {cos_numerical:.10f}",
            f"swept rk4 = {np.degrees(swept_numerical):.6f} deg",
        ],
    )

    # Trajectory in the perifocal frame (p toward perigee, q along the motion).
    p_hat = e_vec / e
    q_hat = np.cross(h_hat, p_hat)
    semilatus = h**2 / MU_EARTH

    def perifocal(angles: np.ndarray) -> np.ndarray:
```

```
275         radius = semilatus / (1.0 + e * np.cos(angles))
276         return np.array([radius * np.cos(angles), radius * np.sin(angles)]) / 1e3
277 
278     fig, ax = plt.subplots(figsize=(5.4, 5.0))
279     ax.add_patch(
280         plt.Circle((0.0, 0.0), R_EARTH / 1e3, facecolor=GRID, edgecolor=BASELINE)
281     )
282     ax.text(0.0, -3.2, "Earth", ha="center", va="center", color=INK_SECONDARY)
283     branch = perifocal(np.radians(np.linspace(-70.0, 70.0, 400)))
284     ax.plot(*branch, color=MUTED, lw=1.0, ls="--", label="hyperbolic trajectory")
285     arc = perifocal(np.linspace(theta0, theta, 200))
286     ax.plot(*arc, color=SERIES[0], lw=2.5, label="10 min coast")
287     for vector, text, offset in (
288         (r0_vec, r"$\mathbf{r}_0$ ($t = 0$, perigee)", (6, -12)),
289         (r_vec, r"$\mathbf{r}$ ($t = 10$ min)", (6, 4)),
290     ):
291         tip = np.array([vector @ p_hat, vector @ q_hat]) / 1e3
292         ax.annotate(
293             "",
294             xy=tip,
295             xytext=(0.0, 0.0),
296             arrowprops={"arrowstyle": "-|>", "color": INK_SECONDARY, "lw": 1.2},
297         )
298         ax.annotate(text, tip, xytext=offset, textcoords="offset points", color=INK)
299     wedge = np.linspace(theta0, theta, 100)
300     ax.plot(8.0 * np.cos(wedge), 8.0 * np.sin(wedge), color=SERIES[1], lw=1.5)
301     middle = (theta0 + theta) / 2.0
302     ax.text(
303         10.9 * np.cos(middle),
304         10.9 * np.sin(middle),
305         rf"$\Delta\theta = {np.degrees(dtheta):.2f}^\circ$",
306         color=INK,
307         ha="left",
308         va="center",
309     )
310     ax.set_aspect("equal")
311     ax.set_xlim(-8.0, 16.0)
312     ax.set_ylim(-14.0, 14.0)
313     ax.set_xlabel(r"$\hat{\mathbf{p}}$ (toward perigee) [$10^3$ km]")
314     ax.set_ylabel(r"$\hat{\mathbf{q}}$ [$10^3$ km]")
315     ax.legend(loc="lower left")
316     save_figure(fig, "01a")
317 
318 
319 def problem_02() -> None:
320     """Curtis 7.7: CW propagation of a satellite near a space station."""
```

```
    heading("p02")
    period = 90.0 * 60.0  # s
    t = 15.0 * 60.0  # s
    n = TAU / period
    dr0 = np.array([1.0, 0.0, 0.0])  # km
    dv0 = np.array([0.0, 10.0, 0.0]) / 1e3  # km/s

    phi_rr, phi_rv, _, _ = cw_stm(n, t)
    dr, dv = cw_propagate(n, t, dr0, dv0)
    distance = np.linalg.norm(dr)
    error = check_cw(n, t, dr0, dv0)

    write_output(
        "02",
        [
            f"n = {n:.10f} rad/s",
            f"nt = {n * t:.10f} rad",
            f"Phi_rr_11 = {phi_rr[0, 0]:.6f}",
            f"Phi_rr_21 = {phi_rr[1, 0]:.6f}",
            f"Phi_rv_12 = {phi_rv[0, 1]:.6f} s",
            f"Phi_rv_22 = {phi_rv[1, 1]:.6f} s",
            *vector_lines("dr", dr, "km"),
            f"distance = {distance:.6f} km",
            *vector_lines("dv", dv * 1e3, "m/s"),
            f"rk4 state difference = {error:.3e}",
        ],
    )

    times = np.linspace(0.0, t, 301)
    path = np.array([cw_propagate(n, tau, dr0, dv0)[0] for tau in times])
    fig, ax = plt.subplots(figsize=(4.6, 5.4))
    ax.plot(path[:, 1], path[:, 0], color=SERIES[0], lw=2.0)
    ax.plot([0.0, dr[1]], [0.0, dr[0]], color=INK_SECONDARY, lw=1.0, ls="--")
    ax.text(
        dr[1] / 2.0 + 0.3,
        dr[0] / 2.0,
        f"{distance:.2f} km",
        color=INK,
        ha="left",
        va="center",
    )
    ax.plot(0.0, 0.0, marker="s", color=INK, ms=7, ls="none")
    ax.annotate(
        "space station",
        (0.0, 0.0),
        xytext=(-8, -4),
```

```
367         textcoords="offset points",
368         ha="right",
369         va="top",
370         color=INK,
371     )
372     ax.plot(dr0[1], dr0[0], "o", mfc="white", mec=SERIES[0], mew=2.0, ms=8)
373     ax.annotate(
374         r"$t = 0$",
375         (dr0[1], dr0[0]),
376         xytext=(-8, 0),
377         textcoords="offset points",
378         ha="right",
379         va="center",
380         color=INK,
381     )
382     ax.plot(dr[1], dr[0], "o", color=SERIES[0], ms=8)
383     ax.annotate(
384         r"$t = 15$ min",
385         (dr[1], dr[0]),
386         xytext=(8, 0),
387         textcoords="offset points",
388         ha="left",
389         va="center",
390         color=INK,
391     )
392     ax.set_aspect("equal")
393     ax.set_xlim(-4.0, 6.0)
394     ax.set_ylim(-1.0, 12.5)
395     ax.set_xlabel(r"$\delta y$, along-track [km]")
396     ax.set_ylabel(r"$\delta x$, radial [km]")
397     save_figure(fig, "02")
398
399
400 def problem_03() -> None:
401     """Curtis 7.8: retrofire by a leading spacecraft in the same circular orbit."""
402     heading("p03")
403     period = 2.0 * 3600.0  # s
404     t = 30.0 * 60.0  # s
405     n = TAU / period
406     dr0 = np.array([0.0, 6.0, 0.0])  # km
407     # Co-orbiting, so B is at rest in A's CW frame until the in-track retrofire.
408     dv0 = np.array([0.0, 0.0, 0.0]) + np.array([0.0, -3.0, 0.0]) / 1e3  # km/s
409
410     phi_rr, phi_rv, phi_vr, phi_vv = cw_stm(n, t)
411     dr, dv = cw_propagate(n, t, dr0, dv0)
412     distance = np.linalg.norm(dr)
```

```
    speed = np.linalg.norm(dv)
    error = check_cw(n, t, dr0, dv0)
    # Before the burn, B is an equilibrium of the CW equations.
    np.testing.assert_allclose(
        cw_propagate(n, t, dr0, np.zeros(3))[0], dr0, rtol=0.0, atol=1e-12
    )

    write_output(
        "03",
        [
            f"n = {n:.10f} rad/s",
            f"nt = {n * t:.10f} rad",
            f"Phi_rr_22 = {phi_rr[1, 1]:.6f}",
            f"Phi_rv_12 = {phi_rv[0, 1]:.6f} s",
            f"Phi_rv_22 = {phi_rv[1, 1]:.6f} s",
            f"Phi_vr_12 = {clean(phi_vr[0, 1]):.6f} 1/s",
            f"Phi_vr_22 = {clean(phi_vr[1, 1]):.6f} 1/s",
            f"Phi_vv_12 = {phi_vv[0, 1]:.6f}",
            f"Phi_vv_22 = {clean(phi_vv[1, 1]):.6f}",
            *vector_lines("dr", dr, "km"),
            f"distance = {distance:.6f} km",
            *vector_lines("dv", dv * 1e3, "m/s"),
            f"speed = {speed * 1e3:.6f} m/s",
            f"rk4 state difference = {error:.3e}",
        ],
    )

    times = np.linspace(0.0, t, 301)
    path = np.array([cw_propagate(n, tau, dr0, dv0)[0] for tau in times])
    fig, ax = plt.subplots(figsize=(5.4, 4.6))
    ax.plot(path[:, 1], path[:, 0], color=SERIES[0], lw=2.0)
    ax.plot([0.0, dr[1]], [0.0, dr[0]], color=INK_SECONDARY, lw=1.0, ls="--")
    ax.annotate(
        f"{distance:.2f} km",
        (dr[1] / 2.0, dr[0] / 2.0),
        xytext=(-8, -8),
        textcoords="offset points",
        color=INK,
        ha="right",
        va="top",
    )
    ax.plot(0.0, 0.0, marker="s", color=INK, ms=7, ls="none")
    ax.annotate(
        r"$A$",
        (0.0, 0.0),
        xytext=(-8, 0),
```

```
459         textcoords="offset points",
460         ha="right",
461         va="center",
462         color=INK,
463     )
464     ax.plot(dr0[1], dr0[0], "o", mfc="white", mec=SERIES[0], mew=2.0, ms=8)
465     ax.annotate(
466         r"$B$, $t = 0$ (retrofire)",
467         (dr0[1], dr0[0]),
468         xytext=(0, 9),
469         textcoords="offset points",
470         ha="center",
471         va="bottom",
472         color=INK,
473     )
474     ax.plot(dr[1], dr[0], "o", color=SERIES[0], ms=8)
475     ax.annotate(
476         r"$B$, $t = 30$ min",
477         (dr[1], dr[0]),
478         xytext=(-10, -6),
479         textcoords="offset points",
480         ha="right",
481         va="top",
482         color=INK,
483     )
484     # Relative velocity at 30 min, drawn at 0.25 km per m/s.
485     arrow = 0.25 * dv * 1e3
486     ax.annotate(
487         "",
488         xy=(dr[1] + arrow[1], dr[0] + arrow[0]),
489         xytext=(dr[1], dr[0]),
490         arrowprops={"arrowstyle": "-|>", "color": SERIES[1], "lw": 1.8},
491     )
492     ax.annotate(
493         rf"$\delta\mathbf{{v}}$, {speed * 1e3:.2f} m/s",
494         (dr[1] + arrow[1], dr[0] + arrow[0]),
495         xytext=(6, 0),
496         textcoords="offset points",
497         ha="left",
498         va="center",
499         color=INK,
500     )
501     ax.set_aspect("equal")
502     ax.set_xlim(-1.5, 13.0)
503     ax.set_ylim(-9.0, 1.5)
504     ax.set_xlabel(r"$\delta y$, along-track [km]")
```

```
505     ax.set_ylabel(r"$\delta x$, radial [km]")
506     save_figure(fig, "03")
507
508
509 def problem_04() -> None:
510     """Curtis 7.10: relative speed half an orbit after an along-track launch."""
511     heading("p04")
512     # Nondimensional: n = 1 and v = 1, so positions are in v/n and time is nt.
513     n, v = 1.0, 1.0
514     dr0 = np.zeros(3)
515     dv0 = np.array([0.0, v, 0.0])
516
517     dr, dv = cw_propagate(n, np.pi / n, dr0, dv0)
518     ratio = np.linalg.norm(dv) / v
519     np.testing.assert_allclose(dv, [0.0, -7.0 * v, 0.0], atol=1e-12)
520     check_cw(n, np.pi / n, dr0, dv0)
521
522     angles = np.linspace(0.0, np.pi, 301)
523     states = [cw_propagate(n, angle / n, dr0, dv0) for angle in angles]
524     path = np.array([state[0] for state in states]) * n / v
525     speeds = np.array([np.linalg.norm(state[1]) for state in states]) / v
526     closed_form = np.sqrt(13.0 - 24.0 * np.cos(angles) + 12.0 * np.cos(angles) ** 2)
527     np.testing.assert_allclose(speeds, closed_form, rtol=1e-12)
528     assert np.argmax(speeds) == len(angles) - 1
529
530     write_output(
531         "04",
532         [
533             f"nt = {np.pi:.10f} rad",
534             *vector_lines("dr", dr * n / v),
535             *vector_lines("dv", dv / v),
536             f"speed ratio = {ratio:.6f}",
537         ],
538     )
539
540     fig, (left, right) = plt.subplots(1, 2, figsize=(8.0, 3.4), width_ratios=(1.4, 1.0))
541     left.plot(path[:, 1], path[:, 0], color=SERIES[0], lw=2.0)
542     left.plot(0.0, 0.0, "o", mfc="white", mec=SERIES[0], mew=2.0, ms=8)
543     left.annotate(
544         r"$nt = 0$, $\delta\mathbf{v}_0 = v\,\hat{\mathbf{j}}$",
545         (0.0, 0.0),
546         xytext=(-6, -8),
547         textcoords="offset points",
548         ha="right",
549         va="top",
550         color=INK,
```

```
    )
    end = dr * n / v
    left.plot(end[1], end[0], "o", color=SERIES[0], ms=8)
    left.annotate(
        r"$nt = \pi$",
        (end[1], end[0]),
        xytext=(0, 9),
        textcoords="offset points",
        ha="center",
        va="bottom",
        color=INK,
    )
    # Relative velocity at nt = pi, drawn at 0.5 (v/n) per v.
    tip = (end[1] + 0.5 * dv[1] / v, end[0] + 0.5 * dv[0] / v)
    left.annotate(
        "",
        xy=tip,
        xytext=(end[1], end[0]),
        arrowprops={"arrowstyle": "-|>", "color": SERIES[1], "lw": 1.8},
    )
    left.annotate(
        r"$\delta\mathbf{v} = -7v\,\hat{\mathbf{j}}$",
        tip,
        xytext=(0, -8),
        textcoords="offset points",
        ha="left",
        va="top",
        color=INK,
    )
    left.set_aspect("equal")
    left.set_xlim(-13.5, 1.5)
    left.set_ylim(-2.0, 5.5)
    left.set_xlabel(r"$n\,\delta y / v$, along-track")
    left.set_ylabel(r"$n\,\delta x / v$, radial")

    right.plot(angles, speeds, color=SERIES[0], lw=2.0)
    right.plot(np.pi, ratio, "o", color=SERIES[0], ms=8)
    right.annotate(
        rf"$|\delta\mathbf{{v}}| = {ratio:.0f}v$",
        (np.pi, ratio),
        xytext=(-6, 6),
        textcoords="offset points",
        ha="right",
        va="bottom",
        color=INK,
    )
```

```
597     right.set_xticks([0.0, np.pi / 2.0, np.pi], ["0", r"$\pi/2$", r"$\pi$"])
598     right.set_ylim(0.0, 8.0)
599     right.set_xlabel(r"$nt$ [rad]")
600     right.set_ylabel(r"$|\delta\mathbf{v}| / v$")
601     fig.tight_layout()
602     save_figure(fig, "04")
603
604
605 def main() -> None:
606     #########
607     # p01 #
608     #########
609     problem_01()
610
611     #########
612     # p02 #
613     #########
614     problem_02()
615
616     #########
617     # p03 #
618     #########
619     problem_03()
620
621     #########
622     # p04 #
623     #########
624     problem_04()
625
626
627 if __name__ == "__main__":
628     main()
```